Lecture 7 · Wed 10/21/2026

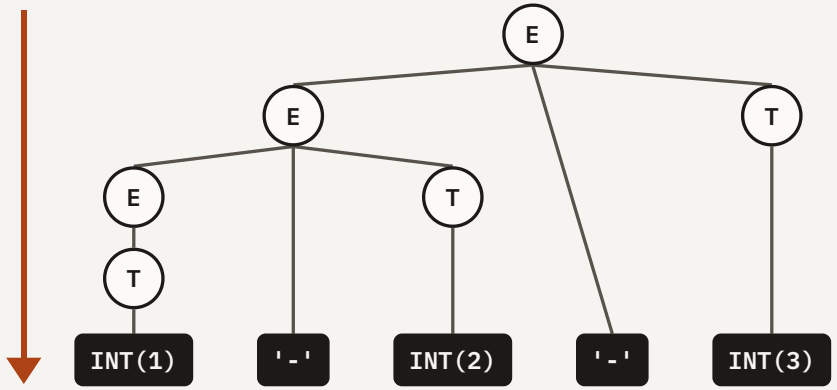
CS160 Compilers — Fall 2026

Parsing II.

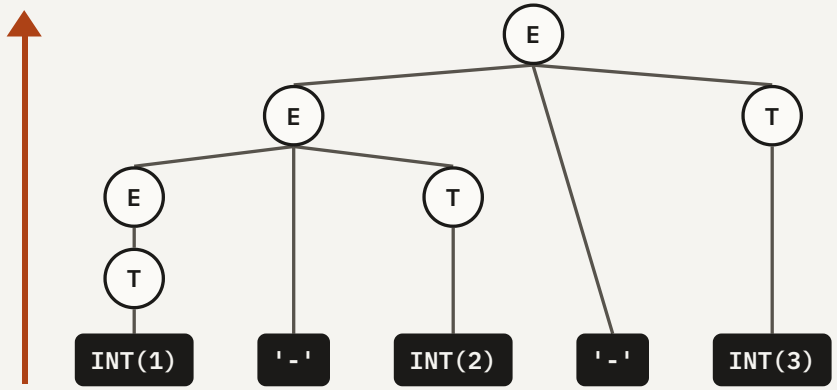
Yu Feng

Two ways to parse: top-down (LL) and bottom-up (LR).

`top-down: root → leaves`



`bottom-up: leaves → root`



top-down

start at the root — the start symbol — and grow the tree toward the leaves, choosing a production at each nonterminal. Lecture 6's recursive descent is one of these. The family is called **LL**: read the input **L**eft to right, produce a **L**eftmost derivation

bottom-up

start at the leaves — the tokens — and build upward, replacing a right-hand side by its left-hand side until only the start symbol is left. The family is called **LR**: **L**eft to right, producing a **R**ightmost derivation in reverse.

One language all lecture — subtraction, `1 - 2 - 3` — and one grammar for it, in two forms. The top-down half uses the form Lecture 6 rewrote it into; the bottom-up half uses the form Lecture 6 started with.

The grammar.

Lecture 6's grammar with '*' and parentheses dropped, left recursion removed as on Lecture 6 slide 16

```
E  -> T E'
E' -> '-' T E'  |  ε
T  -> INT
```

E' appends '-' T as many times as wanted; ε is ‘append nothing’. Tokens of 1 - 2 - 3: INT '-' INT '-' INT.

The one decision a top-down parser makes, and the name LL(1).

```
T  -> INT              only one production: nothing to decide

E' -> '-' T E'  |  ε   next token '-' → first.   When do we choose ε?   ε begins with no token
```

predictive parser

a top-down parser that chooses the production by looking at the next token, without trying alternatives

LL(1)

Left to right, **L**eftmost derivation, **1** token of lookahead. Lecture 6's recursive descent is an LL(1) parser.

Every step of a top-down parser is one decision: at nonterminal A, seeing token a, which production?

Two questions settle it. *Which tokens can begin each alternative?* And, for an alternative that can be empty, *which tokens can follow **A**?* — choosing ε means A is finished, so the token we see belongs to whatever comes after A. The next two slides compute these two sets.

FIRST: which tokens can begin it.

FIRST(X) the tokens that can be the first token of something derived from X. If X can derive the empty string, ε is in FIRST(X) too, and X is **nullable**.

the rules

```
X is a token      FIRST(X) = { X }
X -> Y1 Y2 ...    add FIRST(Y1) to FIRST(X);
                  if Y1 is nullable, also add FIRST(Y2);  and so on
X -> ε            add ε to FIRST(X)
```

our grammar · verified

```
FIRST(T)  = { INT }       T is INT
FIRST(E)  = { INT }       E begins with T
FIRST(E') = { '-', ε }    E' begins with '-', or is empty
```

the grammar

```
E  -> T E'
E' -> '-' T E'  |  ε
T  -> INT
```

FOLLOW: which tokens can come after it.

FOLLOW(A) the tokens that can appear immediately *after* an A in some derivation. **$** is the end-of-input marker, a name for *nothing comes next*.

the rules

```
$ is in FOLLOW(start symbol)
B -> ... A rest              add FIRST(rest) minus ε to FOLLOW(A)   what can begin rest can follow A
  and if rest is nullable:   add FOLLOW(B) to FOLLOW(A)             rest can vanish, so A can end B
B -> ... A                   add FOLLOW(B) to FOLLOW(A)             A ends B
repeat until no set changes
```

the grammar

```
E  -> T E'
E' -> '-' T E'  |  ε
T  -> INT
```

our grammar · verified

```
FOLLOW(E)  = { $ }          start symbol
FOLLOW(E') = { $ }          E' ends E, and ends E'
FOLLOW(T)  = { '-', $ }     E -> T E': E' begins with '-'; E' can vanish, so FOLLOW(E) too
```

To find what follows A, find every place A appears on a right-hand side and look at what is to its right: if something, its FIRST; if nothing, or something that can vanish, whatever follows the whole left-hand side.

The LL(1) parse table.

parse table `M[A, t]`

one row per nonterminal `A`, one column per token `t` (and $); the cell holds the production to use when the parser is at `A` and the next token is `t`. The cell shows the production's right-hand side; the row already names its left-hand side.

the grammar

```
E  -> T E'
E' -> '-' T E' | ε
T  -> INT
```

FIRST (slide 5)

```
FIRST(E)  = { INT }
FIRST(E') = { '-', ε }
FIRST(T)  = { INT }
```

FOLLOW (slide 6)

```
FOLLOW(E)  = { $ }
FOLLOW(E') = { $ }
FOLLOW(T)  = { '-', $ }
```

how each cell was filled

```
E -> T E'       FIRST(T E') = { INT }
E' -> '-' T E'  FIRST = { '-' }
E' -> ε         FOLLOW(E') = { $ }
T -> INT        FIRST = { INT }
```

verified · an empty cell is a syntax error

	INT	'-'	$
E	T E'		
E'		'-' T E'	ε
T	INT		

how to fill it

```
for each production  A -> rhs:
  for every token t in FIRST(rhs):
    write rhs in cell M[A, t]
  if rhs is nullable:
    for every token t in FOLLOW(A):
      write rhs in cell M[A, t]
```

Four productions, four cells. Read `M[E, INT] = T E'` as: *at* E, *seeing* INT, *use* `E -> T E'`. The ε cell is the only one filled by FOLLOW: at `E'`, seeing $, use `E' -> ε`. An empty cell is a syntax error. **A grammar is LL(1) exactly when every cell has at most one entry.** This one is.

Lecture 6's `E -> E '-' T | T` would put both productions in `M[E, INT]` — both begin with `INT` — so it is not LL(1). That is the table's view of the loop Lecture 6 met.

Parsing 1 - 2 - 3 with the table.

stack top at the left · verified

stack	input	action	
E $	1 - 2 - 3 $	M[E, INT] = T E'	replace E
T E' $	1 - 2 - 3 $	M[T, INT] = INT	replace T
INT E' $	1 - 2 - 3 $	INT matches INT	pop both
E' $	- 2 - 3 $	M[E', '-'] = '-' T E'	replace E'
'-' T E' $	- 2 - 3 $	'-' matches '-'	pop both
T E' $	2 - 3 $	M[T, INT] = INT	
INT E' $	2 - 3 $	match	
E' $	- 3 $	M[E', '-'] = '-' T E'	
'-' T E' $	- 3 $	match	
T E' $	3 $	M[T, INT] = INT	
INT E' $	3 $	match	
E' $	$	M[E', $] = ε	pop E'
$	$	accept	

the table from slide 7

	INT	'-'	$
E	T E'		
E'		'-' T E'	ε
T	INT		

Two moves. **Nonterminal on top:** look up `M[top, next token]` and replace the top by the right-hand side. **Token on top:** it must equal the next input token; pop both.

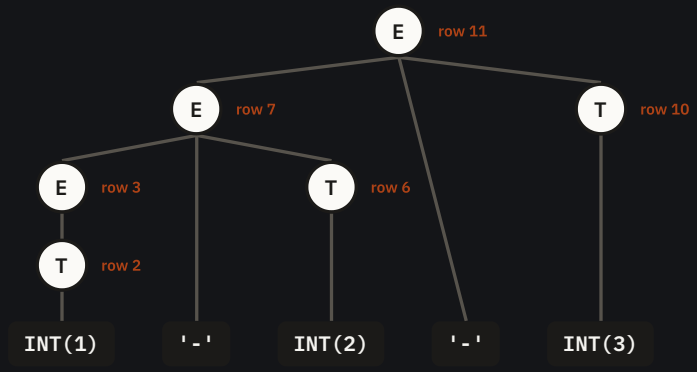
This stack is Lecture 6's call stack written out — `E` above `E' $` is `E()` having called `T()` with the loop still to run. Same parser, no procedures.

Shift-reduce parsing of 1 - 2 - 3.

the same language, in Lecture 6's original left-recursive form · verified

```
E -> E '-' T  |  T
T -> INT
```

	stack	input	action
1		1 - 2 - 3 $	shift INT
2	INT	- 2 - 3 $	reduce T -> INT
3	T	- 2 - 3 $	reduce E -> T
4	E	- 2 - 3 $	shift '-'
5	E '-'	2 - 3 $	shift INT
6	E '-' INT	- 3 $	reduce T -> INT
7	E '-' T	- 3 $	reduce E -> E '-' T
8	E	- 3 $	shift '-'
9	E '-'	3 $	shift INT
10	E '-' INT	$	reduce T -> INT
11	E '-' T	$	reduce E -> E '-' T
12	E	$	accept



row n = the reduce that built the node

the stack

the part of the input read so far, with every completed right-hand side already replaced by its left-hand side. Starts empty; done when it holds exactly the start symbol and the input is used up

shift

take the next input token, push it onto the stack

reduce by `A -> rhs`

the top of the stack reads `rhs`: pop those symbols, push A. One tree node is built.

Row 2: INT is a right-hand side, so it becomes T. Row 3: T is a right-hand side, so it becomes E. Row 7: E `'-'` T is a right-hand side, so it becomes E. The tree grows from the leaves up — and it is Lecture 6's left-leaning tree, from the left-recursive grammar as written.

What the parser remembers: how far along each production is.

The stack told us. E on the stack (row 4): we have the first symbol of E -> E '-' T and still need '-' T — keep reading. E '-' T on the stack (row 7): we have all of it — reduce.

So the parser remembers, for each production it might be in the middle of, how far along it is. Write that with a dot.

item

a production with a dot · in its right-hand side: left of the dot is on the stack, right of the dot is still to come. The dot is a bookmark, not a grammar symbol.

one production, four stages

E -> · E '-' T	stack holds nothing of it yet	
E -> E · '-' T	stack holds E; '-' T still to come	← row 4
E -> E '-' · T	stack holds E '-'; T still to come	← row 5
E -> E '-' T ·	stack holds the whole right-hand side: reduce	← row 7

state

the set of items that match what is on the stack — everything the parser could be in the middle of right now. Dot at the end of an item: reduce is possible. Dot before a token: shifting that token is possible.

slide 9's trace · rows 4, 5, 7 highlighted

	stack	input	action
1		1 - 2 - 3 $	shift INT
2	INT	- 2 - 3 $	reduce T -> INT
3	T	- 2 - 3 $	reduce E -> T
4	**E**	- 2 - 3 $	shift '-'
5	**E '-'**	2 - 3 $	shift INT
6	E '-' INT	- 3 $	reduce T -> INT
7	**E '-' T**	- 3 $	reduce E -> E '-' T
8	E	- 3 $	shift '-'
9	E '-'	3 $	shift INT
10	E '-' INT	$	reduce T -> INT
11	E '-' T	$	reduce E -> E '-' T
12	E	$	accept

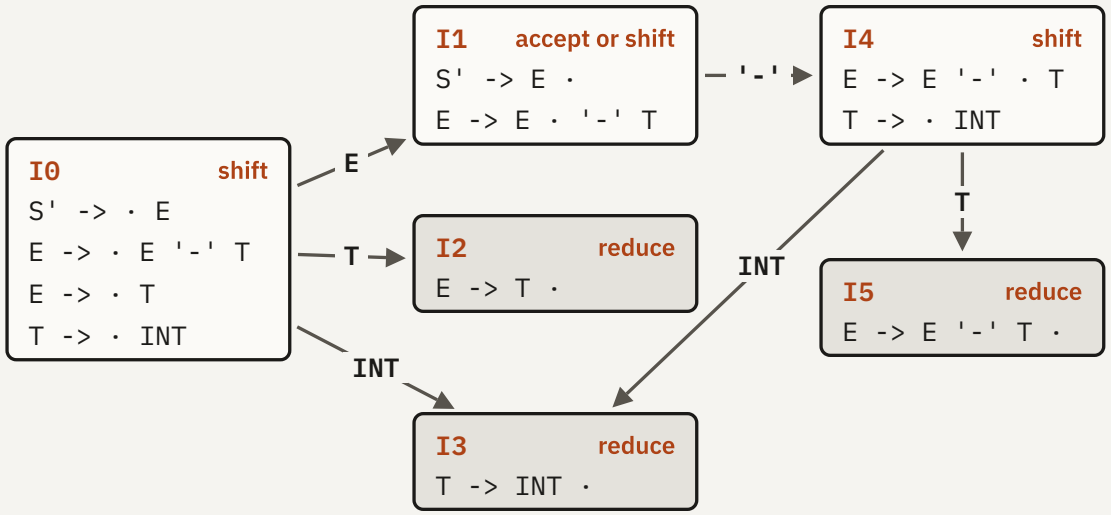
The states, and the state machine.

one line per stack on slide 9 · verified

stack	state	so the parser can
(empty)	I0	shift INT
INT	I3	reduce T -> INT
T	I2	reduce E -> T
E	I1	accept, or shift '-'
E '-'	I4	shift INT
E '-' T	I5	reduce E -> E '-' T

the items of each state are in its box. A new rule S' -> E makes "finished" an item too.

shaded = reduce only



An arrow means: push this symbol, go to that state. A reduce has no arrow: it pops the right-hand side (back to the state before it was pushed), then pushes the left-hand side and follows *that* arrow. Row 2 of slide 9: pop INT, back in I0, push T, I0 → I2.

I1 is the one state with two options — finished, or shift '-'. The next token decides.

The table: the state machine, written as rows and columns.

SLR table · verified

	INT	'-'	$	E	T
I0	shift, go to I3			go to I1	go to I2
I1		shift, go to I4	accept		
I2		reduce E -> T	reduce E -> T		
I3		reduce T -> INT	reduce T -> INT		
I4	shift, go to I3				go to I5
I5		reduce E -> E '-' T	reduce E -> E '-' T		

SLR

Simple **LR**: the table built from slide 11's states, with one refinement — reduce `A -> rhs` only when the next token is in FOLLOW(A). Here FOLLOW(E) = FOLLOW(T) = { `'-'`, `$` }.

how the table is filled, from slide 11

```
state I has an arrow  I --t--> J  for a token t          cell [I, t] = shift, go to J
state I has an item with the dot at the end, A -> rhs ·  cell [I, t] = reduce A -> rhs   for every token t in FOLLOW(A)
state I has  S' -> E ·                                   cell [I, $] = accept
state I has an arrow  I --A--> J  for a nonterminal A    cell [I, A] = go to J          used right after a reduce to A
```

textbooks abbreviate: `s3` = shift, go to I3; `r(E -> T)` = reduce E -> T; `acc` = accept; a bare `2` in the E / T columns = go to I2

Arrows become shifts, dot-at-end items become reduces: the table is slide 11's state machine, one row per state. Row I1: on `'-'` shift to I4; on $ accept — slide 9's question, answered by one token of lookahead, used *after* the right-hand side has been read. This is what `yacc`, `bison` and Lark compute from a grammar; a fifteen-line driver runs it and replays slide 9's twelve rows.

A conflict: one state, one next token, two legal actions.

Lecture 6's ambiguous grammar · verified

```
E -> E '-' E  |  INT    FOLLOW(E) = { '-', $ }

stack  E '-' E,  next token '-':
     E -> E '-' E ·     reduce  →  (1 - 2) - 3
     E -> E · '-' E     shift   →  1 - (2 - 3)
```

conflict

a table cell with two actions in it; the parser cannot run until it is resolved.

The two actions are the two trees of Lecture 6, slide 9. When a tool reports a conflict it is saying: this grammar is ambiguous, or one token is not enough. Use Lecture 6's stratified grammar and the cell is filled once.

Top-down and bottom-up, side by side.

	top-down (LL)	bottom-up (LR)
decides	which production, at its START	which production, at its END
using	the next token: FIRST, FOLLOW	the state (what is on the stack) + the next token
left recursion	must be removed first (Lecture 6, slide 16)	fine as written (slide 9)
grammars handled	fewer	more: every LL(1) grammar and many that are not
written by hand	yes: recursive descent	no: the table comes from a tool
the tool	—	yacc, bison, Lark, … (they build LALR, a compact LR(1))

Both look one token ahead. They differ in *when* they commit. Hand-written compilers are top-down; generated ones are bottom-up.

Monday; PA2 and PA3.

Monday: functions — what the machine does when one piece of code calls another.

PA2 is due tonight; **PA3** is out: functions (`def`, `return`, calls) and a type checker, due Fri 10/30. Its grammar is PA2's statement grammar plus `def` and `return`; a block is `NEWLINE INDENT stmt+ DEDENT`, with the two tokens made by Lecture 5's lexer.

Thursday's section is the PA3 kickoff.